API PHP — routes et utilisation

L'API utilise MongoDB et échange des objets JSON. Les réponses suivent le format `{"code": 200, "response": ...}`. Les identifiants MongoDB sont renvoyés sous forme de chaînes et les dates au format ISO 8601.

Démarrage

Prérequis : PHP 8.1 ou supérieur, l'extension PHP `mongodb`, Composer et un accès à un déploiement MongoDB prenant en charge les transactions (replica set ou cluster shardé). Copiez `.env.example` vers `.env`, puis renseignez les identifiants :

```
cp .env.example .env
# Modifiez .env pour renseigner MONGODB_URI et MONGODB_DATABASE.
php -S 127.0.0.1:8000 index.php
```

Le fichier `.env` est chargé automatiquement par `model/EnvLoader.php`. Les variables déjà définies dans l'environnement du serveur restent prioritaires. Ne partagez pas `.env` et ne le committez jamais.

Le serveur de développement sert alors les routes sous `http://127.0.0.1:8000/api`. En production, configurez le serveur web pour transmettre les chemins `/api/...` à `index.php` et servez l'API exclusivement en HTTPS.

Les collections utilisées sont `Admin`, `Responsables`, `Membres`, `Poles`, `Presences` et `Tokens`. Le compte administrateur est créé directement sur le serveur, hors de l'API. Son champ `password` doit contenir un hash PHP produit par `password_hash($motDePasse, PASSWORD_DEFAULT)`. Les comptes responsables sont créés par un administrateur authentifié via l'API.

Authentification

À l'exception de `POST /api/auth/login/`, toutes les routes nécessitent un jeton Bearer obtenu à la connexion :

```
Authorization: Bearer <jeton>
```

Le jeton aléatoire expire après huit heures. Seul son hash est conservé dans la collection `Tokens` ; les mots de passe des comptes créés par l'API sont hachés avant leur stockage.

Connexion

`POST /api/auth/login/` — administrateur et responsables.

```
{
  "username": "mon-identifiant",
  "password": "mon-mot-de-passe",
  "role": "responsable"
}
```

`role` est facultatif si l'identifiant ne désigne qu'un seul compte. Il peut valoir `admin` ou `responsable`. La réponse contient `token`, `token_type`, `expires_at` et les informations publiques du compte.

Création d'un compte responsable

`POST /api/auth/signin/` — administrateur authentifié uniquement. Le membre indiqué doit exister et ne pas déjà avoir de compte responsable actif. L'administrateur peut choisir un membre de n'importe quel pôle.

```
{
  "username": "nouveau-responsable",
  "password": "un-mot-de-passe-long",
  "membre_id": "66a000000000000000000001"
}
```

Le mot de passe doit comporter au moins 5 caractères. Cette route crée un compte, elle ne remplace pas la connexion. Le responsable pourra ensuite modifier son propre nom d'utilisateur et/ou son mot de passe avec `PUT /api/auth/users/{idUser}` après s'être connecté.

Routes

Toutes les routes ci-dessous sont préfixées par `/api`.

Membres

Méthode	Route	Accès	Description
`GET`	`/membres/`	Admin, responsables	Liste des membres actifs. Un responsable ne voit que son pôle.
`GET`	`/membres/{idMembre}`	Admin, responsables	Détail d'un membre autorisé.
`POST`	`/membres/`	Admin, responsables	Crée un membre dans un pôle existant.
`PATCH`	`/membres/{idMembre}`	Admin, responsables	Modifie `nom` et/ou `prenom` ; seul l'admin peut déplacer un membre avec `pole_id`.
`DELETE`	`/membres/{idMembre}`	Admin	Archive le membre sans supprimer l'historique.

Les nouveaux membres ont `archived: false` et `archived_date: null`. Lorsqu'un membre est archivé, `archived_date` contient la date UTC de l'archivage.

Exemple de création :

```
{
  "nom": "Dupont",
  "prenom": "Aïcha",
  "pole_id": "66a000000000000000000010"
}
```

Pôles

Méthode	Route	Accès	Description
`GET`	`/poles/`	Admin, responsables	Liste des pôles visibles et nombre de membres actifs par pôle.
`GET`	`/poles/{idPole}/membres/`	Admin, responsables	Liste les membres actifs du pôle; un responsable est limité au sien.

Présences

Méthode	Route	Accès	Description
`GET`	`/presences/`	Admin	Liste toutes les séances.
`GET`	`/presences/{idPole}`	Admin, responsable du pôle concerné	Liste les séances du pôle.
`POST`	`/presences/`	Admin, responsables	Enregistre une séance et les membres absents.

La réponse de `GET /api/presences/{idPole}` contient `id_pole` et `nom_pole` une seule fois à la racine, puis la liste `presences`. Chaque séance dans cette liste contient ses champs propres; chaque entrée de `liste_abs` et `auteur` contient `id`, `nom` et `prenom`. Pour un auteur admin, `nom` vaut son username et `prenom` est une chaîne vide. Les anciennes séances dont l'auteur est un membre continuent de renvoyer le nom et le prénom de ce membre.

Exemple de réponse :


```
{
  "code": 200,
  "response": {
    "id_pole": "66a000000000000000000010",
    "nom_pole": "Pôle exemple",
    "presences": [
      {
        "_id": "66a000000000000000000020",
        "date_seance": "2026-10-27T17:30:00+00:00",
        "liste_abs": [
          {
            "id": "66a000000000000000000001",
            "nom": "Dupont",
            "prenom": "Aïcha"
          }
        ],
        "auteur": {
          "id": "66a000000000000000000030",
          "nom": "responsable_test",
          "prenom": ""
        }
      }
    ]
  }
}
```


L'auteur est déterminé automatiquement depuis le compte authentifié, qu'il s'agisse d'un administrateur ou d'un responsable. Ne fournissez pas le champ `auteur` dans la requête. Un responsable ne peut créer une séance que dans son pôle; un administrateur peut agir sur tous les pôles.


```
{
  "id_pole": "66a000000000000000000010",
  "date_seance": "2026-10-27T18:30:00+01:00",
  "liste_abs": [
    "66a000000000000000000001",
    "66a000000000000000000002"
  ]
}
```


L'ID de compte utilisé comme auteur est conservé dans `auteur` pour la traçabilité. La liste `liste_abs` peut être omise ou vide; si elle est renseignée, tous ses membres doivent être actifs et appartenir au pôle de la séance. Le compteur `nbre_abs` de chaque membre absent est incrémenté dans la même transaction que l'enregistrement de

la séance.

Comptes et responsables

Méthode	Route	Accès	Description
`PUT`	`/auth/users/{idUser}`	Admin, responsables	Modifie `username` et/ou `password` ; un responsable ne peut modifier que son compte.
`DELETE`	`/responsables/{idResponsable}`	Admin	Archive un compte responsable.

Les nouveaux comptes responsables ont `archived: false` et `archived_date: null`. Lorsqu'un compte responsable est archivé, `archived_date` contient la date UTC de l'archivage. Ce champ indique quand l'archivage a eu lieu; il ne stocke pas son motif.

Lorsqu'un mot de passe est modifié, les autres jetons de session du compte concerné sont révoqués. Si l'utilisateur modifie son propre mot de passe, le jeton utilisé pour cette requête reste valide; les autres sessions sont déconnectées. Si un administrateur modifie le mot de passe d'un autre compte, tous les jetons de ce compte sont révoqués.

Codes de réponse

Code	Signification
`200`	Requête réussie.
`201`	Ressource créée.
`400`	JSON ou données invalides.
`401`	Jeton absent/invalide ou identifiants incorrects.
`403`	Rôle insuffisant ou accès hors du pôle autorisé.
`404`	Route ou ressource introuvable.
`409`	Conflit, par exemple un nom d'utilisateur déjà utilisé.
`500`	Erreur interne; le détail est consigné côté serveur, pas retourné au client.